

Homework 2

Haozheng Wang

October 2025

1 Harris Corner Detector

1.1 Calculate spatial derivative

Firstly, we convert the RGB picture into gray scale picture using *cv2.cvtColor()*. Then, the picture's original uint8 type is transformed into float64, since we prefer not to calculate gradients in int type. A Gaussian blur follows to remove noise that severely affects derivatives of the image, and finally, we adopt Sobel operation to calculate spatial derivative after all these preparations.

```
1 def gradient_x(img):
2     gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
3     gray_img = np.float64(gray_img)
4     blur_img = ndimage.gaussian_filter(gray_img, sigma=3.0, mode='
        reflect')
5     grad_x = ndimage.sobel(blur_img, axis=1, mode='reflect')
6     return grad_x
7
8
9 def gradient_y(img):
10    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
11    gray_img = np.float64(gray_img)
12    blur_img = ndimage.gaussian_filter(gray_img, sigma=3.0, mode='
        reflect')
13    grad_y = ndimage.sobel(blur_img, axis=0, mode='reflect')
14    return grad_y
```

1.2 Calculate Harris response

The implementation of this part is almost the same as the ideas proposed in page 29. However, it is noteworthy that a Gaussian window is used to reduce noise and enhance the detection of corners here, instead of directly summing up derivatives in a window. And the function *gaussian_blur_kernel_2d()* here is exactly the one that we have implemented in Problem_Set 1.

```
1 def harris_response(img, alpha, win_size):
2     grad_x = gradient_x(img)
```

```

3   grad_y = gradient_y(img)
4   gaussian_kernel = gaussian_blur_kernel_2d(win_size ,
5       standard_deviation=5.0)
6   A = ndimage.convolve(grad_x * grad_x, gaussian_kernel, mode='
7       reflect')
8   B = ndimage.convolve(grad_x * grad_y, gaussian_kernel, mode='
9       reflect')
10  C = ndimage.convolve(grad_y * grad_y, gaussian_kernel, mode='
11      reflect')
12  R = A * C - B ** 2 - alpha * (A + C) ** 2
13  return R

```

1.3 Select candidate corners and non-maximal suppression

It is beneficial to figure out how *ndimage.maximum_filter()* works first:

```

1  # Example
2  matrix = np.array([
3      [1, 2, 3],
4      [4, 5, 6],
5      [7, 8, 9]
6  ])
7  # Use a maximum filter of size 3
8  filtered = ndimage.maximum_filter(matrix, size=3)
9  print(filtered)
10 # Output
11 # [[5, 6, 6],
12 #  [8, 9, 9],
13 #  [8, 9, 9]]

```

As is observed above, local maxima are found with *ndimage.maximum_filter()*, but they are "flattened" into platform areas to some extent. Therefore, we need to locate where those local maxima actually are, and this objective can be achieved by simply comparing the original response array and the filtered array. With regard to the given threshold, responses that are less than this threshold will be set to zero. This should also be taken into consideration when we construct the local maxima condition.

```

1  def corner_selection(R, thresh, min_dist):
2      R_thresh = R.copy()
3      R_thresh[R_thresh < thresh] = 0
4      R_max = ndimage.maximum_filter(R, size=min_dist)
5      is_maxima = (R_max == R_thresh) & (R_thresh != 0)
6      coords = np.argwhere(is_maxima)
7      pix = [(coord[1], coord[0]) for coord in coords]
8      return pix

```

2 Implement Gradient Histogram

We generally follow the 7-step instructions provided, but also make several customized adjustments for efficiency and simplification.

We extract a "region" around the interest point, whose size is $16 \text{ pixels} \times 16 \text{ pixels}$. The region is then divided into 4×4 "blocks" so that each block is $4 \text{ pixels} \times 4 \text{ pixels}$. As the feature vector of the interest point is constructed by combining those 8-bins normalized angle histograms calculated from each block, we will obtain a feature vector of dimension $4 \times 4 \times 8 = 128$ (of course, this refers to the design of SIFT feature).

Moreover, we calculate the principal orientation by constructing an angle histogram on the whole region and extracting the orientation corresponding to the maximum, instead of extracting principal orientation from original feature vector. This practice not only improves the efficiency of the function, but also contributes to higher robustness against extreme values in some block. It is not until the rotation is carried out that we get down to building our feature vector.

```
1 def histogram_of_gradients(img, pix):
2     features = []
3     bins = np.arange(0, 360 + 1, 45)
4     # 1. Calculate gradients
5     grad_x = gradient_x(img)
6     grad_y = gradient_y(img)
7     grad_dir = np.arctan2(grad_y, grad_x) * 180 / np.pi + 180 # np
8     .arctan2: [-pi, pi], converted to [0, 360]
9     grad_mag = np.sqrt(grad_x ** 2 + grad_y ** 2)
10    # 2. Extract the region (4 blocks * 4 blocks, or 16 pixels * 16
11    pixels) around the interest point
12    for (x, y) in pix:
13        interest_point_feature = []
14        region_x_min, region_x_max = max(0, x - 8), min(img.shape
15        [1], x + 8)
16        region_y_min, region_y_max = max(0, y - 8), min(img.shape
17        [0], y + 8)
18        region_dir = grad_dir[region_y_min:region_y_max,
19        region_x_min:region_x_max]
20        region_mag = grad_mag[region_y_min:region_y_max,
21        region_x_min:region_x_max]
22        # 3. Select the prominent gradient of the region and take
23        it as principle orientation
24        region_hist, _ = np.histogram(region_dir, bins=bins,
25        weights=region_mag)
26        principal_orientation = np.argmax(region_hist) * 45
27        # 4. Rotate its neighbor to fit principle orientation
28        region_dir = (region_dir - principal_orientation) % 360
29        # 5. Extract the blocks (4 pixels * 4 pixels)
30        for i in range(4):
31            for j in range(4):
32                block_x_min, block_x_max = 4 * i, min(region_dir.
33                shape[1], 4 * (i + 1))
34                block_y_min, block_y_max = 4 * j, min(region_dir.
35                shape[0], 4 * (j + 1))
36                if block_x_max <= block_x_min or block_y_max <=
37                block_y_min:
```

```

27         interest_point_feature.extend([0 for _ in range
28             (8)])
29         continue
30     block_dir = region_dir[block_y_min:block_y_max,
31         block_x_min:block_x_max]
32     block_mag = region_mag[block_y_min:block_y_max,
33         block_x_min:block_x_max]
34     # 6. For each block, calculate the number of
35     # derivatives in those directions and normalize
36     # the Histogram
37     block_hist, _ = np.histogram(block_dir, bins=bins,
38         weights=block_mag)
39     block_hist /= np.sqrt(np.sum(block_hist ** 2) + 1e
40         -6)
41     interest_point_feature.extend(block_hist)
42     features.extend(interest_point_feature)
43     features = np.array(features).reshape((len(pix), 128))
44     return features

```

3 Local feature matching

We adopt threshold=0.5, and the outputs are as follows:

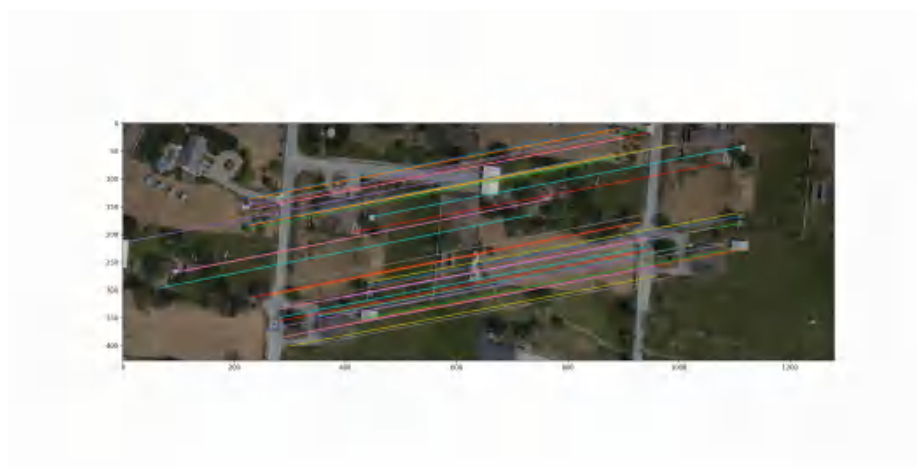


Figure 1: local_feature_matching_1



Figure 2: local_feature_matching_2



Figure 3: local_feature_matching_3

4 Image stitching and Blending

4.1 Compute the alignment of image pairs

We first need to convert the Cartesian coordinates to homogeneous coordinates in order to exert homography matrix:

```

1 def convert_2_homogeneous_coordinates(pixels):
2     n = len(pixels)
3     pixels_array = np.array(pixels)
4     homo_pixels = np.ones((n, 3))

```

```

5 |     homo_pixels[:, 0:2] = pixels_array
6 |     return homo_pixels

```

Then we construct matrix A according to the course materials, which requires some Numpy tricks in indexing and slicing. Our goal is to minimize $\|A_{2n \times 9} h_9\|$ with restriction $\|h\|^2 = 1$, so we find eigenvector of $A^T A$ with the smallest eigenvalue using the Singular Value Decomposition and reshape it to 3×3 matrix.

```

1 | def compute_homography(pixels_1, pixels_2):
2 |     n = len(pixels_1)
3 |     # convert the coordinates to homogeneous coordinates
4 |     homo_pixels_1 = convert_2_homogeneous_coordinates(pixels_1)
5 |     homo_pixels_2 = convert_2_homogeneous_coordinates(pixels_2)
6 |     # form matrix A
7 |     A = np.zeros((2 * n, 9))
8 |     A[0:2*n:2, 0:3] = homo_pixels_1
9 |     A[1:2*n:2, 3:6] = homo_pixels_1
10 |    A[0:2*n:2, 6:9] = -homo_pixels_1 * homo_pixels_2[:, 0:1]
11 |    A[1:2*n:2, 6:9] = -homo_pixels_1 * homo_pixels_2[:, 1:2]
12 |    # use SVD to find the eigenvector of ATA with the smallest
13 |        eigenvalue
14 |    U, S, V = np.linalg.svd((np.transpose(A)).dot(A))
15 |    homo_matrix = np.reshape(V[np.argmin(S)], (3, 3))
    return homo_matrix

```

4.2 Align the image with RANSAC

The RANSAC algorithm introduced in course materials seems too general, and we need to be more specific. The following picture is my handwritten draft, which shows the RANSAC process in this specific problem of calculating the homography matrix:

$$\begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} \approx \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \quad \begin{array}{l} \text{Each pair of points contributes} \\ \text{to 2 equations} \end{array}$$

Degree of freedom: 8

- ① Randomly choose 4 samples, and fit the homography matrix H to those samples.
- ②

(x_1, y_1)
 $\downarrow H$
 $(\hat{x}_1, \hat{y}_1)$
 (x_1', y_1')

(x_2, y_2)
 $\downarrow H$
 $(\hat{x}_2, \hat{y}_2)$
 (x_2', y_2')

$\dots$
 $\downarrow H$
 $\dots$
 $\dots$

(x_n, y_n)
 $\downarrow H$
 $(\hat{x}_n, \hat{y}_n)$
 (x_n', y_n')

Geometric error: $\| (x_i', y_i') - H(x_i, y_i) \|^2$. $\begin{cases} > \text{threshold: outlier} \\ < \text{threshold: inlier} \end{cases}$

- ③ Repeat N times.
- ④ Choose the model that has the largest set of inliers as our final homography matrix $\hat{H}$.

The implementation is therefore straightforward. We suppose that there are 30% inliers, and since the homography matrix needs at least 4 pairs of interest points to fit, we need to iterate at least $N = \lceil \frac{\ln 0.01}{\ln(1-0.3^4)} \rceil = 567$ times to obtain a success probability of at least 99%. That is why we set iter_num=600, slightly larger than $N = 567$.

```

1 def align_pair(pixels_1, pixels_2):
2     n = len(pixels_1)
3     homo_pixels_1 = convert_2_homogeneous_coordinates(pixels_1)
4     homo_pixels_2 = convert_2_homogeneous_coordinates(pixels_2)
5     iter_num = 600
6     threshold = 3.0
7     largest_inliers_num = 0
8     est_homo = np.zeros((3, 3))
9     for i in range(iter_num):
10         # randomly choose 4 samples, and fit the homography matrix
11         # H to those samples
12         random_sample_indices = np.random.choice(n, 4, replace=False)
13         random_pixels_1 = [pixels_1[j] for j in random_sample_indices]
14         random_pixels_2 = [pixels_2[j] for j in random_sample_indices]
15         homo_matrix = compute_homography(random_pixels_1, random_pixels_2)
16         # calculate geometric error and count the inliers
17         pixels_2_tilda = homo_pixels_1.dot(np.transpose(homo_matrix))
18         homo_pixels_2_tilda = pixels_2_tilda / pixels_2_tilda[:, 2:3]

```

```

18     geometric_error = spatial.distance.cdist(
19         homo_pixels_2_tilda, homo_pixels_2, metric='euclidean')
20     geometric_error = np.diagonal(geometric_error)
21     inliers_num = np.sum([geometric_error < threshold])
22     # choose the homo_matrix that has the largest set of
23     # inliers as out final est_homo
24     if inliers_num > largest_inliers_num:
25         largest_inliers_num = inliers_num
26         est_homo = homo_matrix
27     return est_homo

```

4.3 Stitch and blend the image

We choose to follow the pipeline directly, and make a minor adjustment to numerical format so that the code can run in our local environment. For implementations and experiments of advanced blending techniques, please refer to "5.5 Bonus Part II: Advanced Blending Techniques". The outputs of stitch-blend test are as follows (threshold=0.5):



Figure 4: blend_1



Figure 5: blend_2



Figure 6: blend_3

4.4 Generate a panorama

We set $\text{threshold}=0.8$ for this task. All panoramas displayed below are constructed from **five** different images with relatively high quality.

It is noteworthy that we encounter special difficulties when handling the panorama from "library", as the panorama frequently shatters or blows up.

To solve this problem, we carefully select the images to be blended, avoiding images containing large areas of green-colored blocks. However, the success rate is still not high enough, thus Figure 8 is obtained after more than 10 times of experiments and might not be easy to reproduce.



Figure 7: panorama_1



Figure 8: panorama_2



Figure 9: panorama_3



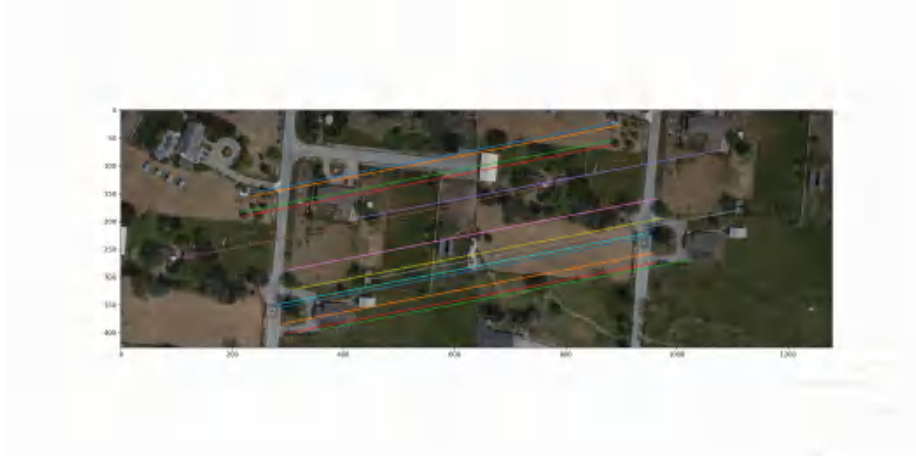
Figure 10: panorama_4

5 Analyze

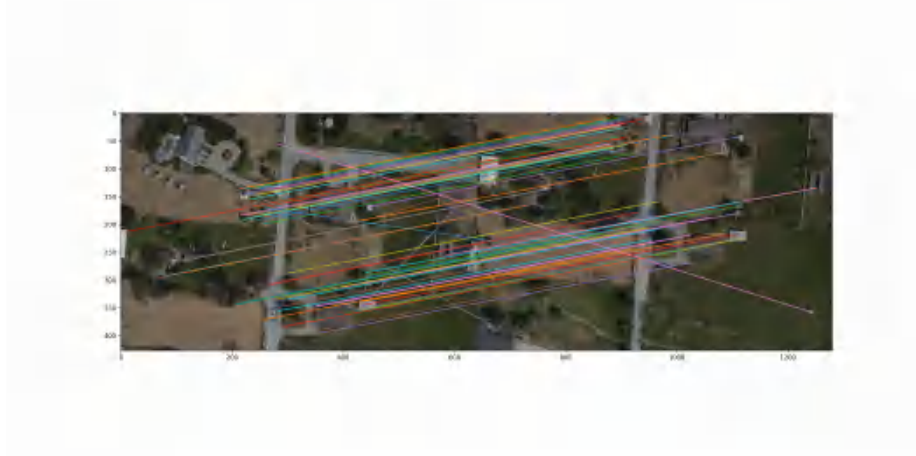
5.1 Hyperparameters

Threshold in "feature_matching": When considering the threshold in feature matching, it is observed that increasing the threshold leads to a rise in

the number of aligned matching pairs (visualized below). The reason is that raising the threshold allows some point pairs with a certain degree of similarity to be considered, which to some extent alleviates the issue of insufficient point pairs that sometimes hinders the estimation of homography matrix. However, increasing the threshold cannot completely solve every problem we encounter, as outliers permitted will cause severe distortion to our panorama.



(a) threshold=0.3



(b) threshold=0.8

Figure 11: Threshold in "feature_matching"

Threshold in "align_pair": This threshold represents a boundary between inliers and outliers. Intuitively speaking, increasing the threshold leads to a more strict constraint on inliers, resulting in fewer number of inliers of a

model. However, this holds true for all models inspected, so as long as the order of magnitude of the threshold stays within a normal range, it has relatively fewer impact on the quality of our panorama. The following visual comparison provides evidence for this assertion.

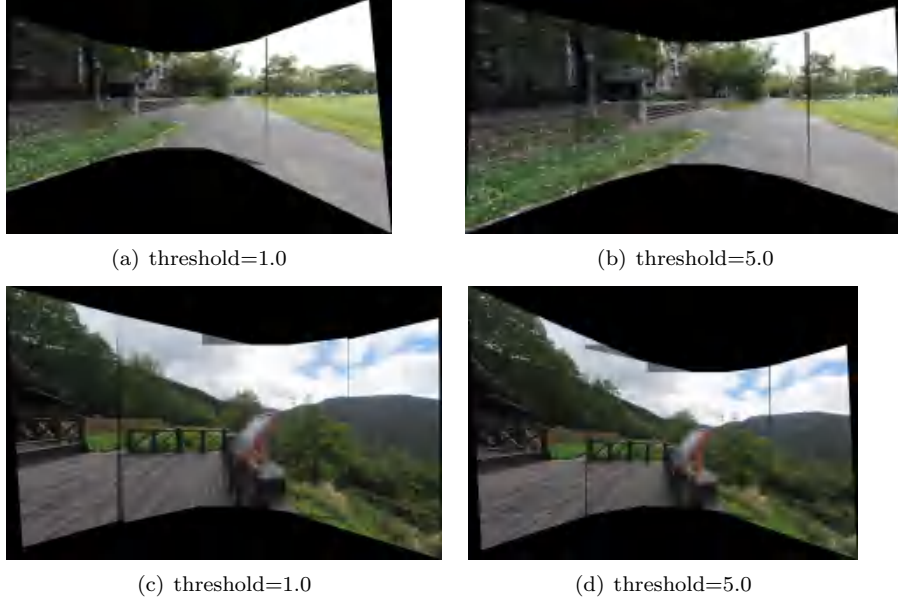


Figure 12: Threshold in "align_pair"

5.2 How to move camera during shooting

We notice that some previous reports have claimed that panoramas generated from rotation images have better quality than those from translation images; here, "rotation images" means that the set of images are photographed by a camera at a fixed point with different orientations, while "translation images" means that the images are photographed by a camera moving on a line with a fixed orientation. They argued that tackling rotation image is somehow equivalent to transform a surface into a plane, thus weakening the effect of homography matrix.

However, what we observe in our experiment tells a different story, as the quality of translation images turns out to be far from that of rotation images with obvious ghosting. With the assistance of a center spot and a white straight line on the playground, we can ensure that both set of images are shot in accordance with the requirements. This phenomena could be explained in terms of parallax error, which is not a severe problem for rotation moving with a shared projection center but poses a threat to translation moving. For academical authenticity, we report this abnormal phenomena with the following two pairs of pictures.



(a) rotation experiment 1



(b) translation experiment 1



(c) rotation experiment 2



(d) translation experiment 2

Figure 13: Comparison between rotating and translating the camera

5.3 How you blend the images

The default sequence of image blending provided is starting from the middle and respectively expanding the panorama to the left and to the right, and we manage to modify this sequence and start blending from the left. However, the number of blended images gets restrained under four, since the geometric error might accumulate rapidly as more images line up to join the principle image from one side directly. Accordingly, the quality of blend-from-left is to some extent lower than that of blend-from-mid.



Figure 14: Comparison between blend-from-mid and blend-from-left

5.4 Bonus Part I: Feature Detectors and Descriptors

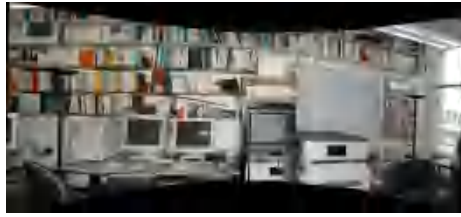
We manage to use more powerful and robust feature detectors and descriptors to improve the panorama, including SIFT, Accelerated-KAZE(AKAZE) and Binary Robust Invariant Scalable Keypoints(BRISK). We directly call the OpenCV API as shown below:

```

1 if mode == 'SIFT':
2     sift = cv2.SIFT_create() # initialize the detector
3     interest_points_1, descriptors_1 = sift.detectAndCompute(img_1,
4         None) # "None" means no mask is used
5     interest_points_2, descriptors_2 = sift.detectAndCompute(img_2,
6         None)
7     # SIFT descriptor is floating-point type, so we use L2-distance
8     # for matching
9     matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)

```

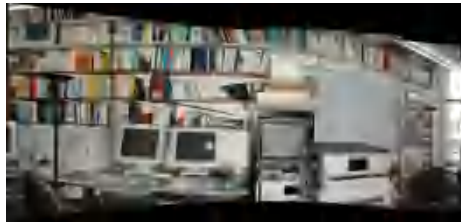
The brute-force matcher is then used for feature matching. With these feature detectors and descriptors, we blend five images into a panorama for each image set, and the results are shown in following figures:



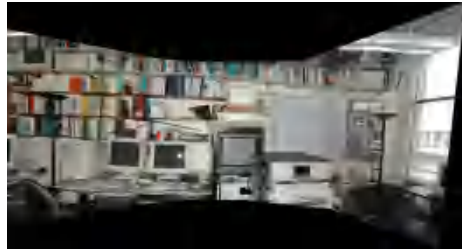
(a) grail_HoG



(b) grail_SIFT



(c) grail_AKAZE



(d) parrington_BRISK

Figure 15: Panoramas from "grail"



(a) library_HoG



(b) library_SIFT

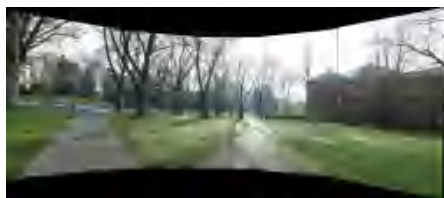


(c) library_AKAZE

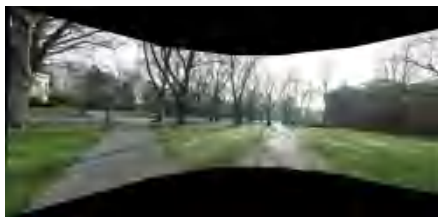


(d) library_BRISK

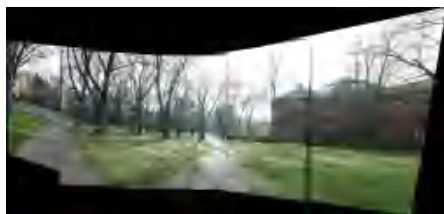
Figure 16: Panoramas from "library"



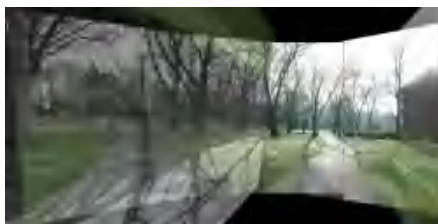
(a) parrington_HoG



(b) parrington_SIFT



(c) parrington_AKAZE



(d) parrington_BRISK

Figure 17: Panoramas from "parrington"



(a) Xue-Mountain-Entrance_HoG



(b) Xue-Mountain-Entrance_SIFT



(c) Xue-Mountain-Entrance_AKAZE



(d) Xue-Mountain-Entrance_BRISK

Figure 18: Panoramas from "Xue-Mountain-Entrance"

It might be noticed that HoG feature has similar performance to SIFT, and sometimes encounters similar difficulties as well. It is unsurprising that the methods perform similarly, as our implementation of HOG owes a significant conceptual debt to the SIFT feature design.

The results indicate that AKAZE and BRISK perform much worse than expected. Neither AKAZE nor BRISK can show improvements on "library", and BRISK even fails to complete "parrington" which HoG and SIFT can carry out with ease. By comparison, SIFT feature turns out to be more powerful and robust (maybe that is why we have laid an emphasis on SIFT in class).

5.5 Bonus Part 2: Advanced Blending Techniques

We follow the pipeline of alpha blending to develop Poisson blending and pyramid blending. Before carrying out these two types of blending, the first thing to do is to determine the left image and the right image; that is, if `est_img_1` is not the left image, we need to exchange `est_img_1` and `est_img_2` together with their boundaries:

```

1     non_zero_1 = np.argwhere(est_img_1 > 0)
2     non_zero_2 = np.argwhere(est_img_2 > 0)
3     left_1, right_1 = np.min(non_zero_1[:, 1]), np.max(non_zero_1
4        [:, 1])
5     left_2, right_2 = np.min(non_zero_2[:, 1]), np.max(non_zero_2
6        [:, 1])
7     if left_1 > left_2:
8         est_img_1, est_img_2 = est_img_2, est_img_1
9         left_1, left_2, right_1, right_2 = left_2, left_1, right_2,
10            right_1

```

To implement pyramid blending, we firstly construct Gaussian Pyramid and Laplacian Pyramid for both images. Then, we blend two Laplacian Pyramids by taking the first half of the left image and the latter half of the right image and join these two halves horizontally in the end. The blended image can thus be reconstructed from the blended Laplacian Pyramid. For more details about this implementation, please refer to my code in "*utils.py*".

Unfortunately, our version of pyramid blending turns out to be incapable of processing more than three images, which is indeed a limitation that cannot be ignored. However, the panoramas generated by pyramid blending with three images are still blessed with some interesting features, including much smoother connection and preliminary ghosting removal. The following figure can illustrate this point fairly well:

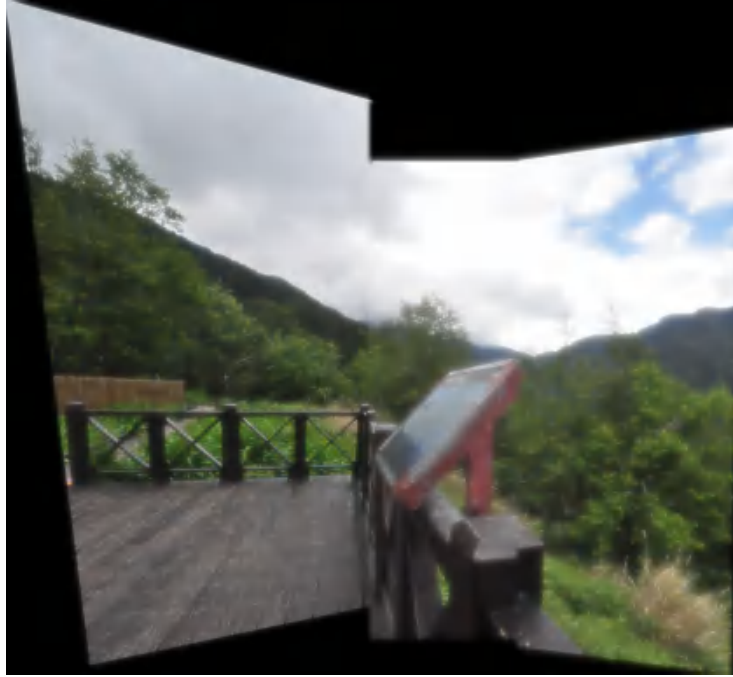
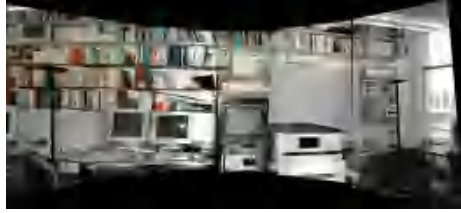


Figure 19:more Poisson than Poisson



Figure 20: More panoramas generated by pyramid blending

Poisson blending can be realised more easily with the assistance of *cv2.seamlessClone()*. Nonetheless, preparing for the parameters of this functions can still be sort of annoying. We need to figure out the source image and the target image along with some descriptions of their properties. It might be noticed that the indices used in our code seem strange due to overlaps and ommissions; However, those designs contribute to tighter connections which can improve blending quality according to our experiments. And the outputs are as follows:



(a) grail_poisson



(b) parrington_poisson



(c) library_poisson



(d) Xue-Mountain-Entrance_poisson

Figure 21: Panoramas generated by Poisson blending

Poisson blending performs well as it effectively conserve details and eliminates ghosting as well. However, the stitching boundaries remain distinct, which indicates that the advantages of Poisson blending are not fully demonstrated with this particular set of input images.